

transcript

QA Transcript — HXC-014

Revision: 1 Created: 2026-05-28 Last modified: 2026-05-28 Status: active

Field	Value
Run-id	HXC-014 (covers HXC-014 batches 1–11 + HXC-014b)
Feature	§11.4.85 stress + chaos test coverage of the in-process Go packages + the systemic i18n translator hardening (HXC-014b)
Type	Task / Bug
Tracker	docs/Issues.md (HXC-014, HXC-014b rows)
Authority	constitution submodule Constitution.md §11.4.85 (stress+chaos) + §11.4.83 (this evidence mandate)
Status	PASS — verified end-to-end under -race

Table of contents

- [Feature exercised](#)
- [Captured evidence — coverage](#)
- [Captured evidence — real bugs fixed \(anti-bluff\)](#)
- [Bidirectional thread](#)

Feature exercised

The end-user-visible guarantee of HXC-014 is **resilience under load + failure injection**: every covered in-process package now ships stress (sustained p50/p95/p99 + ≥ 10 -goroutine concurrency + boundary) and chaos (panic / cancel / input-corruption / resource-pressure / state-corruption) suites that run under the Go race detector with captured evidence. The “user” is a maintainer running `make stress-chaos` before a release to confirm the platform does not crash, deadlock, leak, or race under adversarial conditions.

All outputs below are real captured results from the 2026-05-28 session — no output is fabricated. Per-run JSON evidence (latency.json / concurrency_report.json / recovery_trace.json) is written under `helix_code/qa-results/<run-id>/` (gitignored per CONST-053; the harness writes it on every run).

Captured evidence — coverage

31 in-process packages covered across 11 subagent batches: worker, load_balancer, task, session, event, memory, context, rules, repomap, discovery, tools, workflow, hooks, agent, monitoring, commands, mcp, notification, performance, security, providers, llm (manager), editor, template, cognee, focus, config, persistence, auth, deployment, project.

The maintainer-facing gate:

```
$ cd helix_code && make stress-chaos
🐛 Running §11.4.85 stress + chaos suites (under -race)...
ok dev.helix.code/internal/worker      ...
ok dev.helix.code/internal/event      ...
ok dev.helix.code/internal/memory     ...
ok dev.helix.code/internal/hooks      ...
ok dev.helix.code/internal/agent      ...
ok dev.helix.code/internal/config     ...
... (all 31 covered packages) ...
ok dev.helix.code/tests/stresschaos
✅ Stress + chaos suites complete (evidence under qa-results/)
```

Result: 0 FAIL across the full target under `-race` (re-run by the conductor at each batch close-out). Representative captured latencies: event sustained publish p50=0.008ms p95=0.016ms p99=0.022ms (N=1500); memory create/add/get p50=0.501ms p95=1.133ms p99=1.373ms (N=1200); auth JWT validate p50=0.076ms p95=0.106ms p99=0.130ms (N=500); concurrent runs gDelta=0 deadlock=false throughout.

Captured evidence — real bugs fixed (anti-bluff)

The chaos suites surfaced **29 real production bugs** (full per-bug detail + file:line in the docs/Issues.md HXC-014 row and the per-batch commit bodies). Defect classes:

- **Panic-in-goroutine, no recover()** (process-wide crash): event, hooks, mcp×2, notification, monitoring, commands, security, template, focus, persistence.
- **Non-reentrant RWMutex re-locked under read-lock** (deadlock): worker, hooks, providers.
- **Map/var mutated+read with no/unused mutex** (data race / concurrent-map crash): agent, performance, memory, config (no mutex at all), project, deployment.
- **Security:** auth VerifyJWT panicked on a forged-but-validly-signed token with non-string claims (crash-on-untrusted-input DoS).

Each fix is anti-bluff-proven by §1.1 paired mutation. Example (auth):

```
# revert the fix → forged-claim token crashes the process:
$ go test -race -run TestAuth_Chaos_HostileClaims ./internal/auth
panic: interface conversion: interface {} is float64, not string ... FAIL
# restore the comma-ok guard:
$ go test -race -run TestAuth_Chaos_HostileClaims ./internal/auth
ok dev.helix.code/internal/auth
```

HXC-014b — the same unguarded-translator defect (data race + panic-crash) was copy-pasted across the i18n layer; fixed in all 54 `internal/*/translator.go` seams

```
(translatorMu+recover()), mutation-proven in internal/logging/  
translator_race_test.go:  
  
# mutated (unguarded) seam:  
WARNING: DATA RACE  
panic: HXC-014b: hostile translator panic ... FAIL  
# restored (guarded): ok dev.helix.code/internal/logging 1.0s
```

Bidirectional thread

Exchange 1 — maintainer runs the resilience gate

Sent: `cd helix_code && make stress-chaos`

Received: all 31 covered packages ok under `-race`, 0 FAIL, evidence written to `qa-results/` (see the coverage section above for the captured tail).

Exchange 2 — maintainer asks “do the tests actually catch breakage?”

Sent (for each fix): revert the fix in production code, re-run the matching `-race` chaos test.

Received: the test FAILS with the genuine defect signature (DATA RACE / deadlock-timeout / SIGSEGV / panic); restoring the fix returns OK. This is the §1.1 paired-mutation proof recorded per fix in the per-batch commit bodies.

Conclusion: the resilience guarantee is real and the suites are non-bluffing.